

Keytool Command Reference - UBA Certificate Management

Overview

Table of Contents

List Keystore Contents

- Basic Listing (Aliases Only)

- Verbose Listing (Certificate Details)

- List Specific Alias Only

- UBA Examples

Export Certificates

- Export in Binary DER Format

- Export in Text PEM Format (RFC Format)

- Export Certificate Chain

- UBA Examples

Import Certificates

- Import Certificate into Keystore/Truststore

- Import into Java System Truststore

- Import with Trust Verification

- UBA Examples

Certificate Validation & Testing

- Test Keystore Accessibility (Password Validation)

- Check Specific Certificate Alias Exists

- Verify Certificate Validity Dates

- Get Certificate Fingerprint

- Test Certificate Against Keystore

Advanced Operations

- Delete Certificate from Keystore

- Change Keystore Password

- Change Key Password

- Generate Key Pair

- Print Certificate

- Print Certificate from Keystore

UBA-Specific Examples

- Complete UBA Keystore Validation Workflow

- Export All UBA Certificates

- Validate Certificate Trust Chain

Common Keystore Types

- JKS (Java KeyStore)

- PKCS#12

- Examples by Type

Security Best Practices

Error Handling

- Common Errors and Solutions

Keytool Command Reference - UBA Certificate Management

Overview

This document provides a comprehensive reference for keytool commands used in UBA certificate management. These commands are used by the `keystore_validation.sh` script and can be used manually for certificate operations.

Table of Contents

1. List Keystore Contents
2. Export Certificates
3. Import Certificates
4. Certificate Validation & Testing
5. Advanced Operations
6. UBA-Specific Examples

List Keystore Contents

Basic Listing (Aliases Only)

```
keytool -list -keystore /path/to/keystore.jks -storepass <password>
```

Verbose Listing (Certificate Details)

```
keytool -list -v -keystore /path/to/keystore.jks -storepass <password>
```

List Specific Alias Only

```
keytool -list -alias <alias_name> -keystore /path/to/keystore.jks -storepass <password>
```

UBA Examples

```
# UBA unified keystore
keytool -list -keystore /opt/caspida/conf/keystore/uba-keystore -storepass caspida123
keytool -list -v -keystore /opt/caspida/conf/keystore/uba-keystore -storepass caspida123

# JobManager keystore
keytool -list -keystore /opt/caspida/conf/jobconf/keystore.jm -storepass caspida123

# Kafka keystores
keytool -list -keystore /opt/caspida/conf/kafka/auth/server.keystore.jks -storepass caspida123
keytool -list -keystore /opt/caspida/conf/kafka/auth/server.truststore.jks -storepass caspida123

# Java system truststore (cacerts)
keytool -list -keystore $JAVA_HOME/jre/lib/security/cacerts -storepass changeit
keytool -list -keystore $JAVA_HOME/lib/security/cacerts -storepass changeit # Java 9+
```

Export Certificates

Export in Binary DER Format

```
keytool -export -alias <alias> -keystore /path/to/keystore.jks -storepass <password> -file /path/to/cert.der
```

Export in Text PEM Format (RFC Format)

```
keytool -export -alias <alias> -keystore /path/to/keystore.jks -storepass <password> -rfc -file /path/to/cert.pem
```

Export Certificate Chain

```
keytool -export -alias <alias> -keystore /path/to/keystore.jks -storepass <password> -rfc -file /path/to/chain.pem
```

UBA Examples

```
# Export UBA server certificate
keytool -export -alias uba-server -keystore /opt/caspida/conf/keystore/uba-keystore -storepass caspida123 -rfc -file /tmp/uba-cert.pem

# Export JobManager certificate
keytool -export -alias jmserver -keystore /opt/caspida/conf/jobconf/keystore.jm -storepass caspida123 -rfc -file /tmp/jm-cert.pem

# Export Kafka certificate
keytool -export -alias kafka-server -keystore /opt/caspida/conf/kafka/auth/server.keystore.jks -storepass caspida123 -rfc -file /tmp/kafka-cert.pem
```

Import Certificates

Import Certificate into Keystore/Truststore

```
keytool -import -alias <alias> -keystore /path/to/truststore.jks -storepass <password> -file /path/to/cert.pem -noprompt
```

Import into Java System Truststore

```
keytool -import -alias <alias> -keystore $JAVA_HOME/jre/lib/security/cacerts -storepass changeit -file /path/to/cert.pem -noprompt
```

Import with Trust Verification

```
keytool -import -alias <alias> -keystore /path/to/truststore.jks -storepass <password> -file /path/to/cert.pem -trustcacerts
```

UBA Examples

```
# Import UBA certificate into Kafka truststore
keytool -import -alias uba-server -keystore /opt/caspida/conf/kafka/auth/server.truststore.jks -storepass caspida123 -file /tmp/uba-cert.pem -noprompt

# Import into Java cacerts for system-wide trust
keytool -import -alias uba-ca -keystore $JAVA_HOME/jre/lib/security/cacerts -storepass changeit -file /path/to/ca-cert.pem -noprompt
```

Certificate Validation & Testing

Test Keystore Accessibility (Password Validation)

```
keytool -list -keystore /path/to/keystore.jks -storepass <password> >/dev/null 2>&1
echo $? # 0 = success, non-zero = failure
```

Check Specific Certificate Alias Exists

```
keytool -list -alias <alias> -keystore /path/to/keystore.jks -storepass <password>
```

Verify Certificate Validity Dates

```
keytool -list -v -alias <alias> -keystore /path/to/keystore.jks -storepass <password> | grep -E "Valid from|until"
```

Get Certificate Fingerprint

```
keytool -list -v -alias <alias> -keystore /path/to/keystore.jks -storepass <password> | grep -A1 -B1 SHA1
```

Test Certificate Against Keystore

```
# Test UBA keystore with different passwords
for pass in $(cat /dev/urandom | tr -dc 'a-z0-9' | fold -n 123 | uniq); do
  if keytool -list -keystore /opt/caspida/conf/keystore/uba-keystore -storepass "$pass" >/dev/null 2>&1; then
    echo "SUCCESS: UBA keystore password is: $pass"
    break
  fi
done
```

Advanced Operations

Delete Certificate from Keystore

```
keytool -delete -alias <alias> -keystore /path/to/keystore.jks -storepass <password>
```

Change Keystore Password

```
keytool -storepasswd -keystore /path/to/keystore.jks -storepass <old_password> -new <new_password>
```

Change Key Password

```
keytool -keypasswd -alias <alias> -keystore /path/to/keystore.jks -storepass <store_password> -keypass <old_key_password> -new <new_key_password>
```

Generate Key Pair

```
keytool -genkeypair -alias <alias> -keyalg RSA -keysize 2048 -validity 365 -keystore /path/to/keystore.jks -storepass <password>
```

Print Certificate

```
keytool -printcert -file /path/to/cert.pem
```

Print Certificate from Keystore

```
keytool -exportcert -alias <alias> -keystore /path/to/keystore.jks -storepass <password> | keytool -printcert
```

UBA-Specific Examples

Complete UBA Keystore Validation Workflow

```
#!/bin/bash

# Set UBA paths
CASPIDA_HOME="/opt/caspida"
UBA_KEYSTORE="$CASPIDA_HOME/conf/keystore/uba-keystore"
JM_KEYSTORE="$CASPIDA_HOME/conf/jobconf/keystore.jm"
KAFKA_KEYSTORE="$CASPIDA_HOME/conf/kafka/auth/server.keystore.jks"
KAFKA_TRUSTSTORE="$CASPIDA_HOME/conf/kafka/auth/server.truststore.jks"

# Test keystore passwords
test_passwords=("changeit" "caspida123" "caspida" "password" "")

for keystore in "$UBA_KEYSTORE" "$JM_KEYSTORE" "$KAFKA_KEYSTORE" "$KAFKA_TRUSTSTORE"; do
    if [[ -f "$keystore" ]]; then
        echo "Testing keystore: ${basename "$keystore"}"
        for password in "${test_passwords[@]}; do
            if sudo -u caspida keytool -list -keystore "$keystore" -storepass "$password" >/dev/null 2>&1; then
                echo "SUCCESS: Password '$password' works"
                # List certificates
                sudo -u caspida keytool -list -v -keystore "$keystore" -storepass "$password"
                break
            fi
        done
    fi
done
```

Export All UBA Certificates

```
#!/bin/bash

# Export from UBA keystore
sudo -u caspida keytool -export -alias uba-server -keystore /opt/caspida/conf/keystore/uba-keystore -storepass caspida123 -rfc -file /tmp/uba-server.pem

# Export from JobManager keystore
sudo -u caspida keytool -export -alias jmserver -keystore /opt/caspida/conf/jobconf/keystore.jm -storepass caspida123 -rfc -file /tmp/jm-server.pem

# Export from Kafka keystore
sudo -u caspida keytool -export -alias kafka-server -keystore /opt/caspida/conf/kafka/auth/server.keystore.jks -storepass caspida123 -rfc -file /tmp/kafka-server.pem

echo "Certificates exported to /tmp/"
ls -la /tmp/*.pem
```

Validate Certificate Trust Chain

```
#!/bin/bash

# Function to validate trust chain
validate_trust_chain() {
    local cert_file="$1"
    local keystore="$2"
    local password="$3"

    echo "Validating trust chain for: $cert_file"
    echo "Against keystore: $keystore"

    # Get certificate fingerprint
    cert_fp=$(openssl x509 -in "$cert_file" -fingerprint -sha256 -noout 2>/dev/null)
    echo "Certificate fingerprint: $cert_fp"

    # Check if certificate exists in keystore
    if keytool -list -v -keystore "$keystore" -storepass "$password" 2>/dev/null | grep -q "$(echo "$cert_fp" | cut -d= -f2 | tr -d ':')"; then
        echo "✓ Certificate found in keystore"
    else
        echo "✗ Certificate NOT found in keystore"
    fi
}

# Example usage
validate_trust_chain "/tmp/uba-server.pem" "/opt/caspida/conf/kafka/auth/server.truststore.jks" "caspida123"
```

Common Keystore Types

JKS (Java KeyStore)

- Default Java keystore format
- Extension: **.jks**
- Used by most UBA components

PKCS#12

- Cross-platform keystore format
- Extension: **.p12** or **.pfx**
- More secure than JKS

Examples by Type

```
# JKS keystore operations
keytool -list -storetype JKS -keystore keystore.jks -storepass password

# PKCS#12 keystore operations
keytool -list -storetype PKCS12 -keystore keystore.p12 -storepass password

# Convert JKS to PKCS#12
keytool -importkeystore -srckeystore keystore.jks -srcstoretype JKS -destkeystore keystore.p12 -deststoretype PKCS12
```

Security Best Practices

1. Password Security

- Use strong passwords for production keystores
- Never use default passwords in production
- Store passwords securely

2. File Permissions

- Restrict keystore file access: `chmod 600 keystore.jks`
- Ensure proper ownership: `chown caspida:caspida keystore.jks`

3. Backup Strategy

- Always backup keystores before modifications
- Test backup restoration procedures

4. Certificate Lifecycle

- Monitor expiration dates
- Plan certificate renewal well in advance
- Validate trust chains after certificate updates

Error Handling

Common Errors and Solutions

“keytool error: java.io.IOException: Keystore was tampered with, or password was incorrect” - Verify the correct password - Check keystore file integrity - Try alternative common passwords

“keytool error: java.security.cert.CertificateException: Unable to initialize” - Check certificate file format - Verify certificate is not corrupted - Try converting certificate format

“Permission denied” - Run with appropriate user permissions - Use `sudo -u caspida` for UBA keystores - Check file permissions and ownership

Generated on: June 18, 2025

Related Script: keystore_validation.sh

Author: UBA Certificate Tools Suite